CE-1103 · Lección 03

Estructuras de Datos Lineales

Arreglos, listas, pilas y colas: datos organizados uno tras otro

Lista.java

```
Node n = new Node(7);
n.next = head;
head = n;
size++;
```

Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Describir arreglos y matrices y sus límites
2. Distinguir un TDA de su implementación
3. Implementar listas con arreglos y con nodos enlazados
4. Aplicar LIFO con pilas y FIFO con colas
5. Explicar las colas de prioridad
6. Usar generics para estructuras reutilizables

// introducción

¿Qué es una estructura de datos lineal?

1

Estructura de datos

Forma de organizar datos para usarlos de manera eficiente: procesar, recuperar y almacenar.

2

Lineal

Elementos en secuencia, uno tras otro, en un solo nivel. Se recorren en una sola pasada.

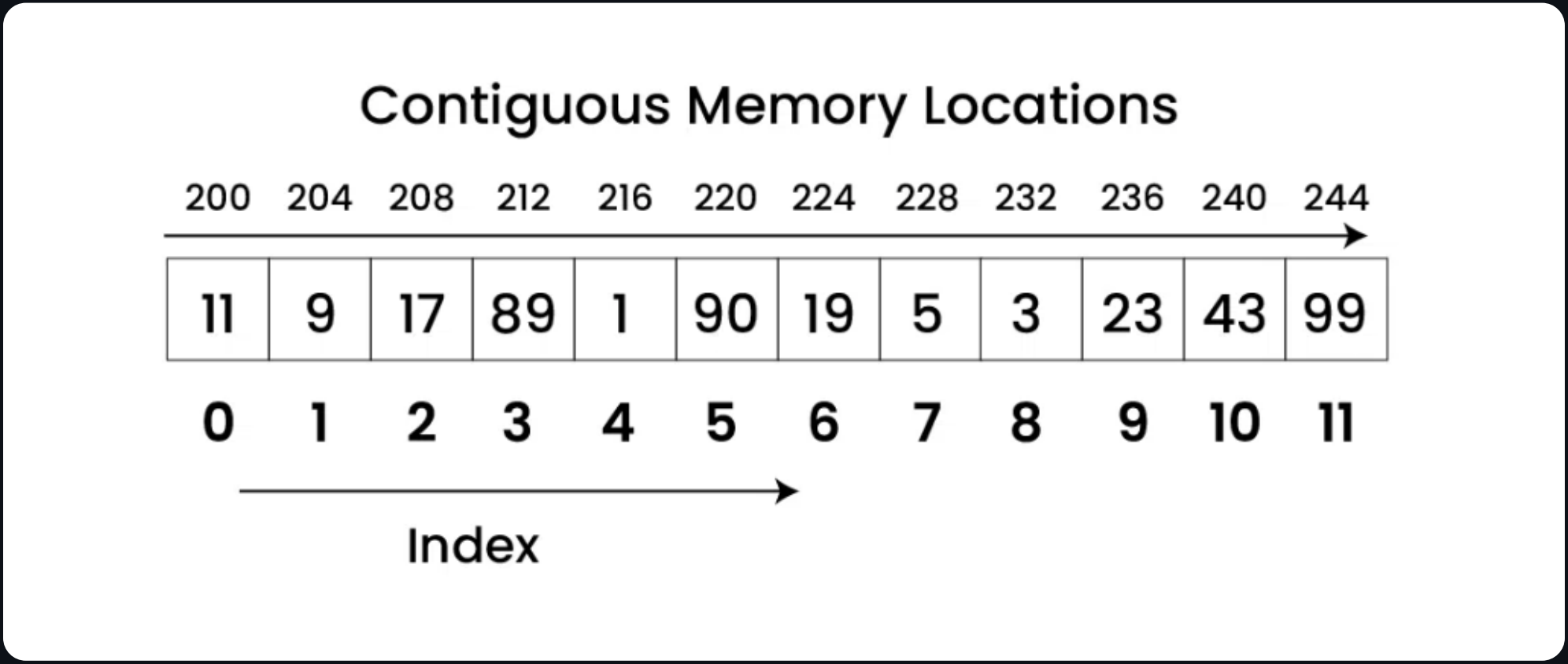
3

Predecesor y sucesor

Cada elemento tiene un predecesor y un sucesor, excepto el primero y el último.

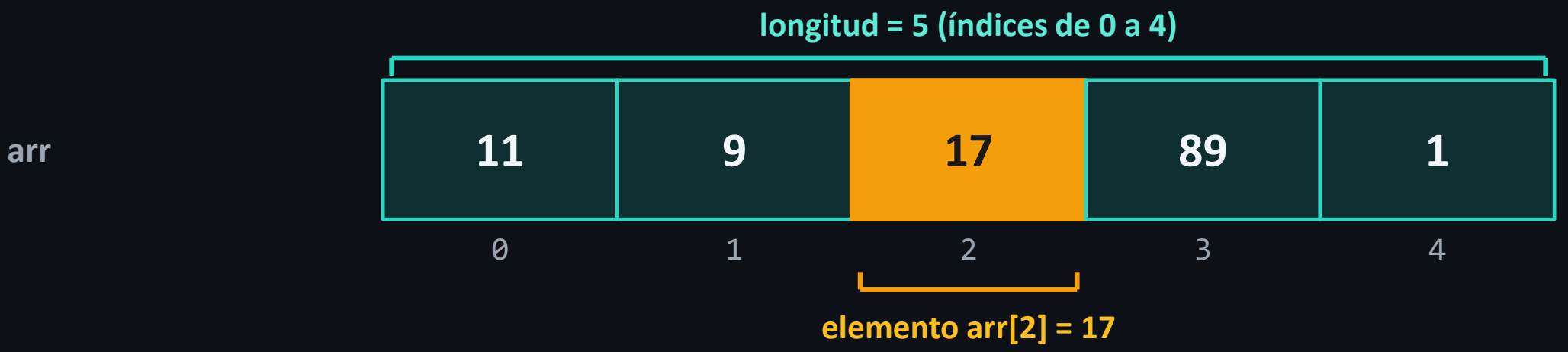
En esta lección: arreglos · listas · pilas · colas · colas de prioridad.

Arreglo: posiciones contiguas de memoria



Mismo tipo y mismo tamaño: un `int` ocupa 4 bytes; las direcciones suben de 4 en 4.

Terminología básica de arreglos



Índice: posición del elemento; empieza en 0.

Longitud: número de elementos del arreglo.

Elemento: valor guardado en una posición.

// arreglos y matrices

Declarar y acceder a un arreglo

Declaración e inicialización

```
int[] arr = new int[5];  // con tamaño
int[] nums = { 1, 2, 3, 4, 5 };  // en línea
char[] letras = { 'a', 'b', 'c' };
float[] reales = { 1.4f, 2.0f, 24f };
```

Acceso por índice

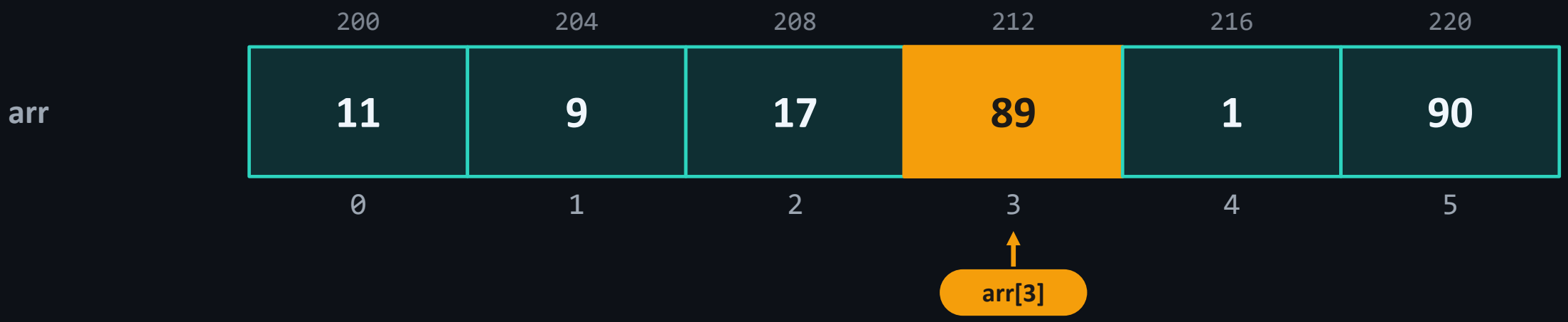
```
arr[0] = 1;
int x = arr[0];
```

Mismo tamaño

Cada posición mide lo mismo según el tipo: `int` y `float` ocupan 4 bytes.

> Para estos ejemplos, la sintaxis en **C#** es idéntica a la de **Java**.

Acceso aleatorio: una suma y un acceso



Dirección: `arr + 3 * sizeof(int)` = 200 + 3 × 4 = **212**.

Una suma y un acceso a memoria: **tiempo constante**, sin importar el tamaño del arreglo.

Ventajas y desventajas de los arreglos

Ventajas

- **Acceso aleatorio** en tiempo constante
- Fácil de implementar y usar
- **Cero overhead** de memoria: no guarda información extra

vs

Desventajas

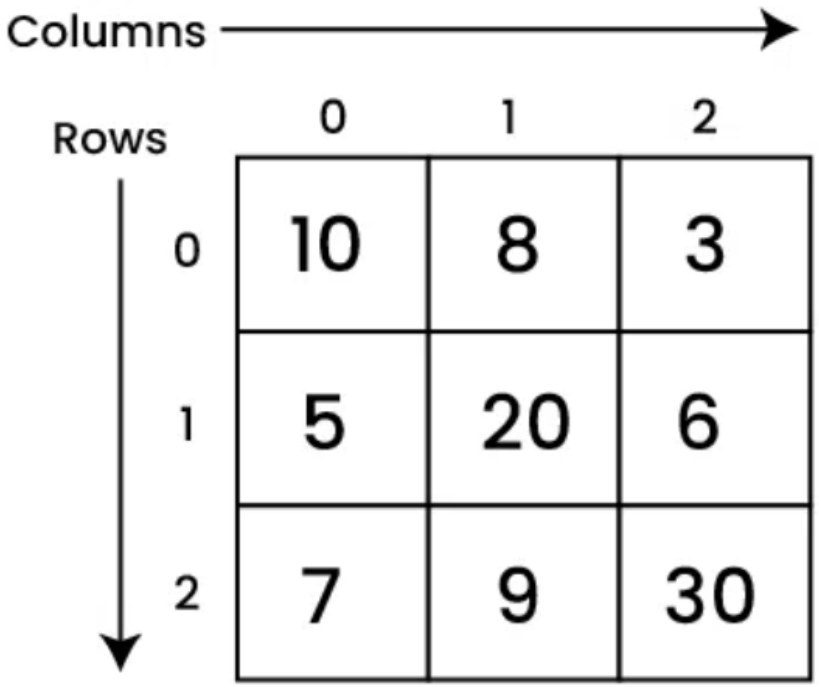
- **Tamaño fijo:** para crecer hay que crear otro arreglo y copiar
- **Insertar y eliminar** obliga a mover elementos

> Las listas resuelven la limitación del tamaño fijo.

Insertar en un arreglo: mover elementos



Matrices: arreglos de dos dimensiones



Una matriz es un arreglo de arreglos: se representa como una tabla de filas y columnas.

// arreglos y matrices

Matrices en Java

Matriz 3×3 inicializada

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Acceso: fila 1, columna 2

```
int x = matriz[1][2];  // 6
```

Declarar con tamaño y asignar

```
int[][] matriz = new int[3][3];
matriz[0][0] = 1;
matriz[0][1] = 2;
```

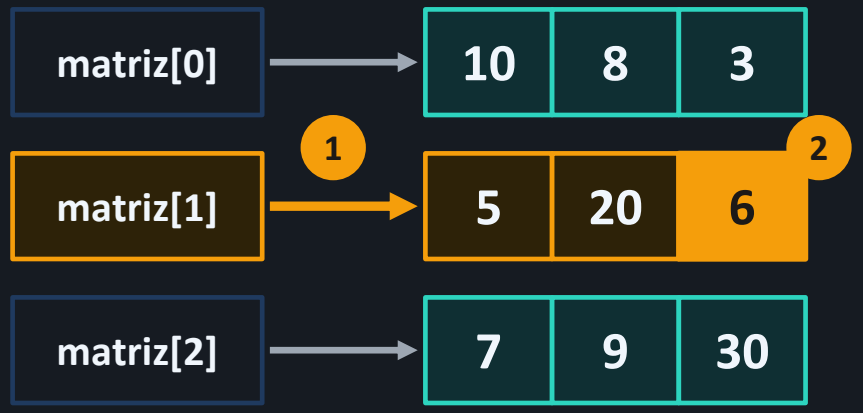
Más dimensiones (3D)

```
int[][][] matriz3D = new int[3][3][3];
matriz3D[0][0][0] = 1;
```

> **C#** ofrece una forma más compacta: `int[,] matriz = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };`

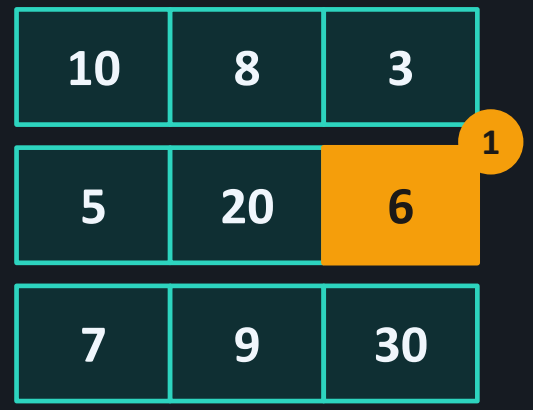
¿Memoria contigua en las matrices?

Java: arreglo de referencias



Dos accesos: `matriz[1]` y luego el elemento.

C#: bloque contiguo



Un acceso a memoria y una suma.

Las matrices **no** siempre se almacenan en memoria contigua: depende del lenguaje.

Arreglo vs. lista

Arreglo

- Tamaño **fijo**
- Posiciones contiguas de memoria
- Acceso directo por índice

vs

Lista

- **Sin tamaño fijo:** se agregan y eliminan elementos
- Respeta el **orden de inserción**
- Varias implementaciones posibles

> La lista es un **tipo de dato abstracto**: define las operaciones, no cómo se implementan.

Tipo de dato abstracto (TDA): el qué, no el cómo

TDA: el qué

Un **conjunto de objetos** junto con un **conjunto de operaciones**.

- No dice **cómo** se implementan las operaciones
- Ejemplos: listas, colas, conjuntos, grafos
- Operaciones típicas: `add`, `remove`, `contains`

vs

Implementación: el cómo

En Java: una **clase** o **interfaz** que oculta los detalles.

- Usar el TDA es **llamar métodos**
- Un mismo TDA admite varias implementaciones
- No se exponen detalles, como la clase `Node`

Decisión de diseño: qué operaciones incluye un TDA lo decide quien lo diseña.

TDA Lista: operaciones básicas

Operación	Método	Qué hace
Obtener	get	Devuelve el elemento en una posición específica
Agregar	add	Añade un elemento a la lista
Eliminar	remove	Elimina un elemento de la lista
Contiene	contains	Verifica si un elemento está en la lista
Tamaño	size	Devuelve el número de elementos de la lista

Los ejemplos también usan `isEmpty()` y `clear()`.

get: con arreglo es `storage[i]`; con nodos se recorre desde `head`.

// listas

Cuatro implementaciones de lista

ArrayList

Implementación mediante **arreglos**.

SinglyLinkedList

Mediante nodos **simplemente enlazados**.

DoubleLinkedList

Mediante nodos **doblemente enlazados**.

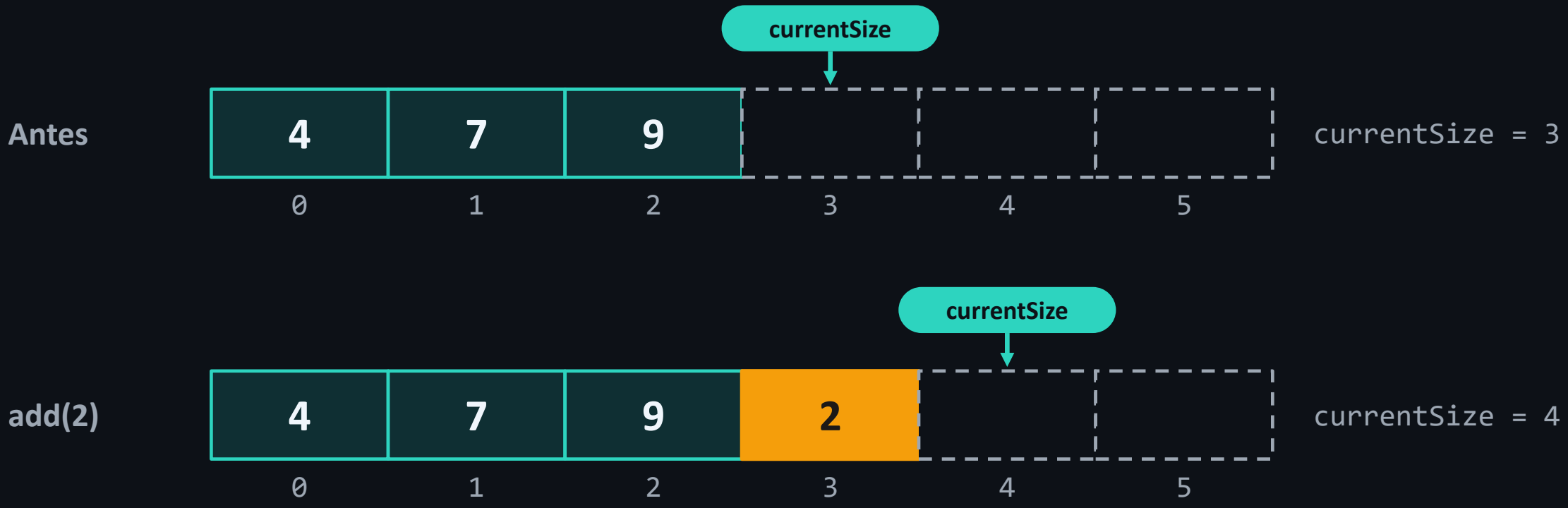
CircularLinkedList

Mediante nodos enlazados **circulares**.

> Mismas operaciones; cambian el **costo** y el uso de **memoria**.

// lista con arreglos

ArrayList: agregar al final



`storage[currentSize++] = element;` escribe y avanza. Capacidad: `maxSize = 6`.

Si se llena: crear un arreglo mayor y copiar los elementos (un «hit» de performance).

// lista con arreglos

ArrayList: eliminar desplaza a la izquierda



Se busca el elemento, se copia cada elemento posterior una posición a la izquierda y se hace `currentSize--`.

// lista con arreglos

ArrayList: add y remove

Estructura y add

```
public class ArrayList implements List {
    private int maxSize, currentSize;
    private int[] storage;

    public boolean add(int element) {
        if (currentSize >= maxSize) {
            return false;
        }
        storage[currentSize++] = element;
        return true;
    }
}
```

remove

```
public int remove(int element) {
    for (int i = 0; i < currentSize; i++) {
        if (storage[i] == element) {
            for (int j = i; j < currentSize - 1; j++)
                storage[j] = storage[j + 1];
            currentSize--;
            return element;
        }
    }
    throw new NoSuchElementException();
}
```

Ventaja: el programador no lidia con «shifts» ni «re-sizes» del arreglo.

Ojo: no es un arreglo estático: es una lista implementada con uno.

// lista simple enlazada

Lista simple: nodos con valor y referencia

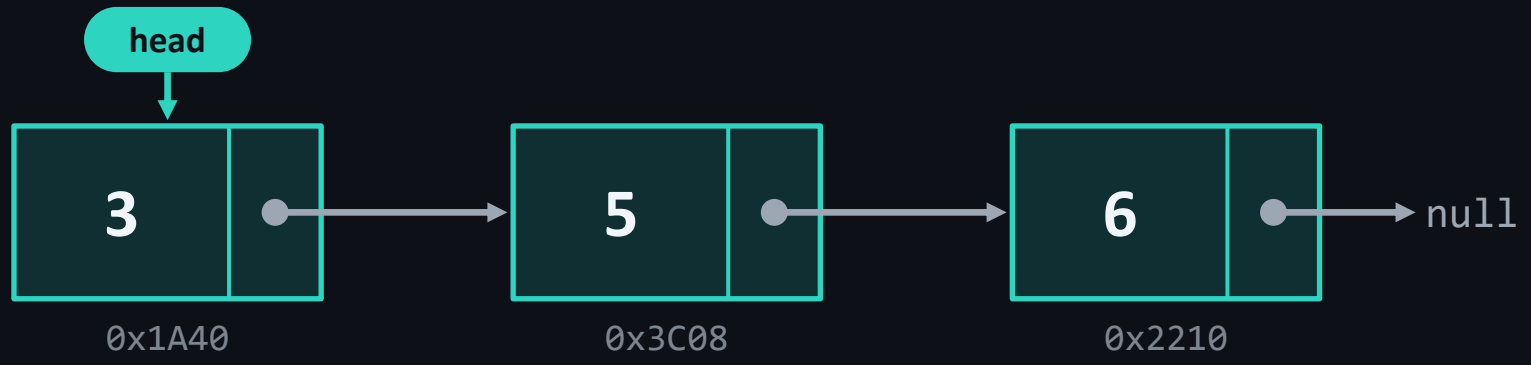


Cada **nodo** guarda un **valor** y una **referencia** al nodo siguiente; el último apunta a `null`.

Opcional: una referencia a `tail` acelera insertar al final (**DoubleEndedLinkedList**).

// lista simple enlazada

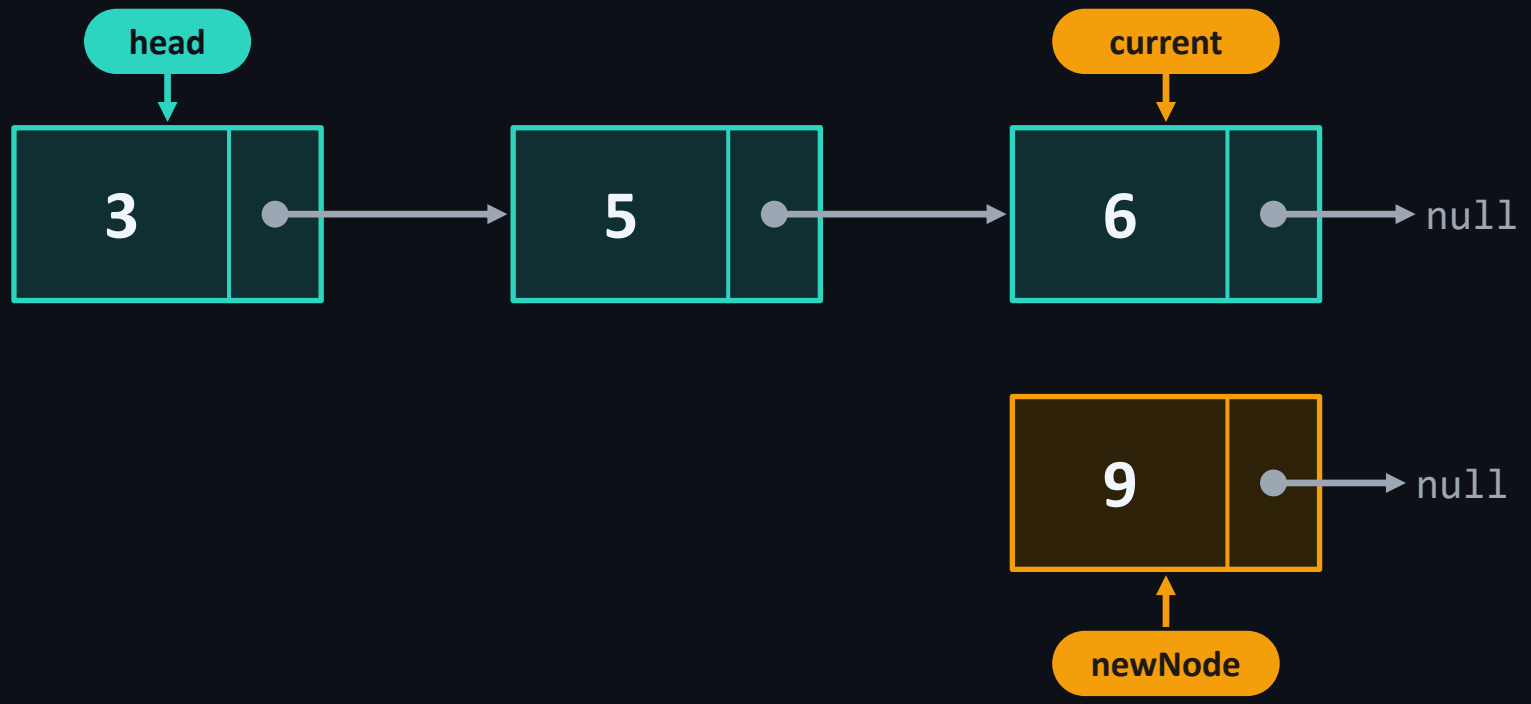
Los nodos viven dispersos en el heap



Cada nodo se crea en el **heap** justo cuando se agrega: las direcciones **no** son contiguas.

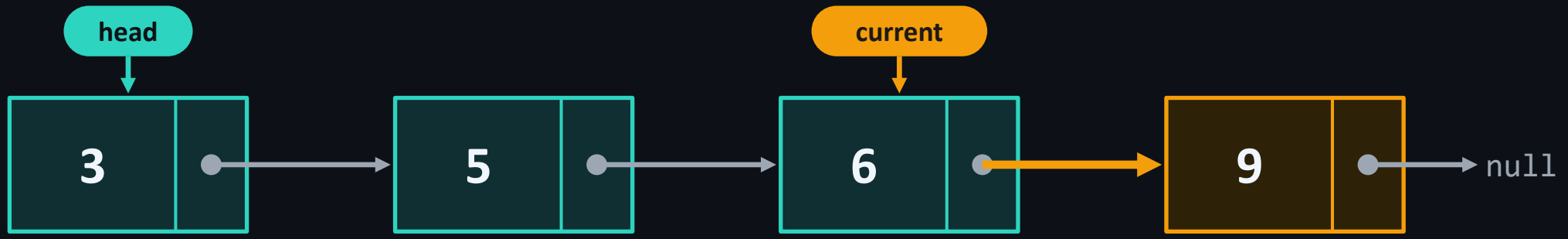
Si se pierde `head`, se pierde toda la lista.

add(9): crear el nodo y recorrer



Se crea `newNode` y `current` avanza hasta el último nodo (`current.next == null`).

add(9): enlazar el nuevo nodo



`current.next = newNode;` y `size++`. El nuevo nodo ya apunta a `null`.

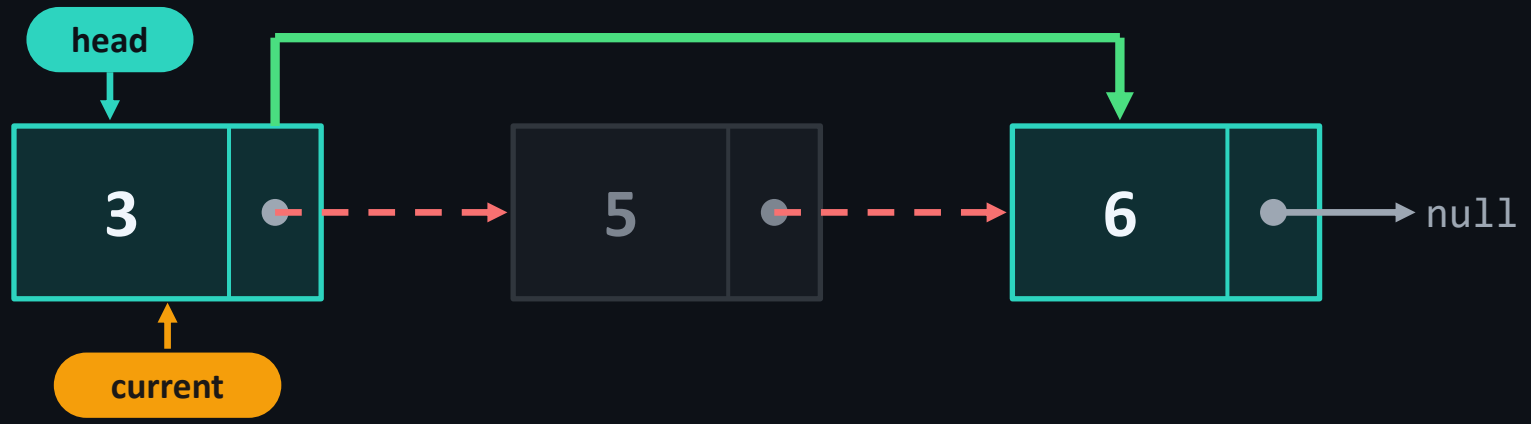
Consejo: dibuje los nodos y las flechas antes de escribir el código.

remove(5): localizar el nodo anterior



Se recorre hasta que `current.next.value == 5`: `current` es el nodo **anterior** al que se elimina.

remove(5): saltar el nodo



`current.next = current.next.next;` El nodo 5 queda sin referencias; se hace `size--`.

// lista simple enlazada

Lista simple: nodo, estructura y contains

Nodo (clase interna del paquete)

```
class Node {
    int value;
    Node next;

    Node(int value) { this.value = value; }
}
```

Lista

```
public class SinglyLinkedList implements List {
    private Node head;
    private int size;
}
```

contains: recorrer hasta encontrar

```
public boolean contains(int element) {
    Node current = head;
    while (current != null) {
        if (current.value == element) {
            return true;
        }
        current = current.next;
    }
    return false;
}
```

// lista simple enlazada

Lista simple: add y remove

add (al final)

```
public boolean add(int element) {
    Node newNode = new Node(element);
    if (head == null) {
        head = newNode;
    } else {
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    size++;
    return true;
}
```

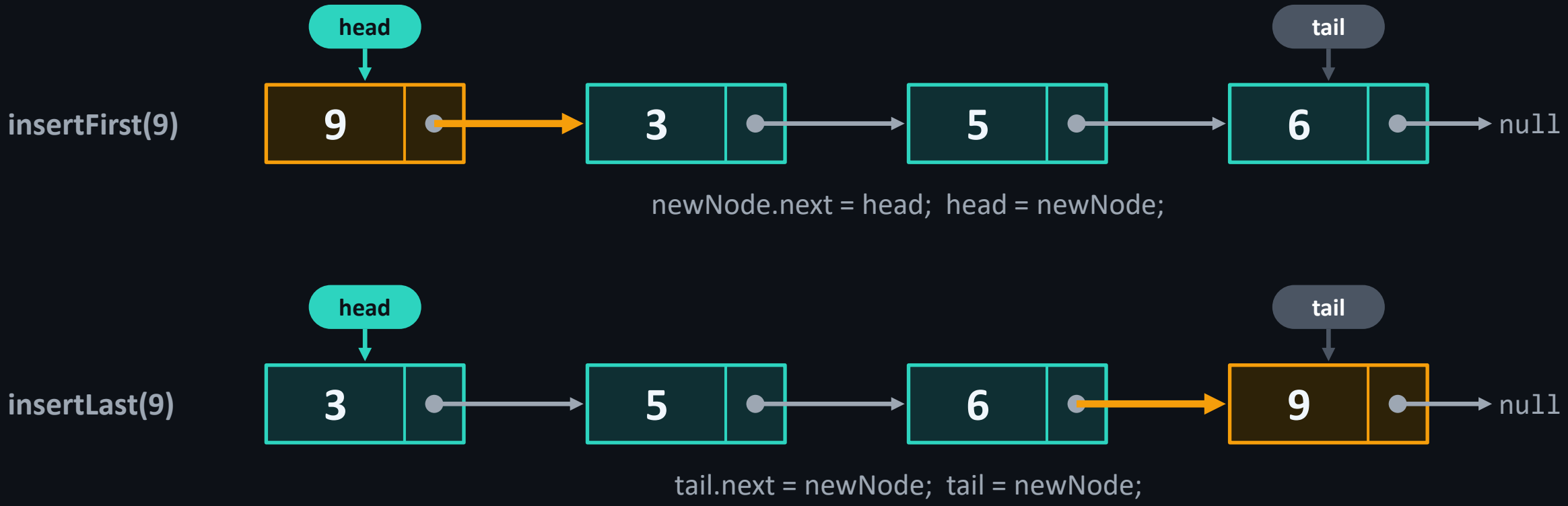
remove: caso general

```
Node current = head;
while (current.next != null) {
    if (current.next.value == element) {
        current.next = current.next.next;
        size--;
        return element;
    }
    current = current.next;
}
throw new NoSuchElementException();
```

Caso especial: la cabeza

Si `head.value == element`: `head = head.next;`

Insertar al inicio y al final sin recorrer



Sin tail: agregar al final recorre toda la lista (n pasos).

Con head y tail: ambos extremos cuestan 2 asignaciones, sin importar n.

// lista doble

Lista doblemente enlazada



Cada nodo guarda una referencia al **anterior** (`prev`) y al **siguiente** (`next`).

Se recorre en **ambos sentidos**; el primero y el último apuntan a `null`.

// lista doble

Lista doble: ventaja, costo y usos

Ventaja

- Recorrido **hacia adelante y hacia atrás**

vs

Costo

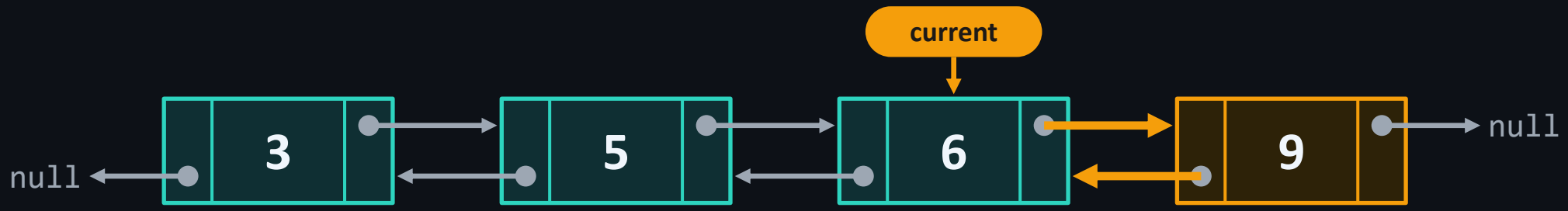
- Una referencia extra por nodo: **más memoria**
- Implementación **más compleja**

Aplicabilidad: editor de texto (adelante y atrás) e historial de navegación del navegador web.

// lista doble

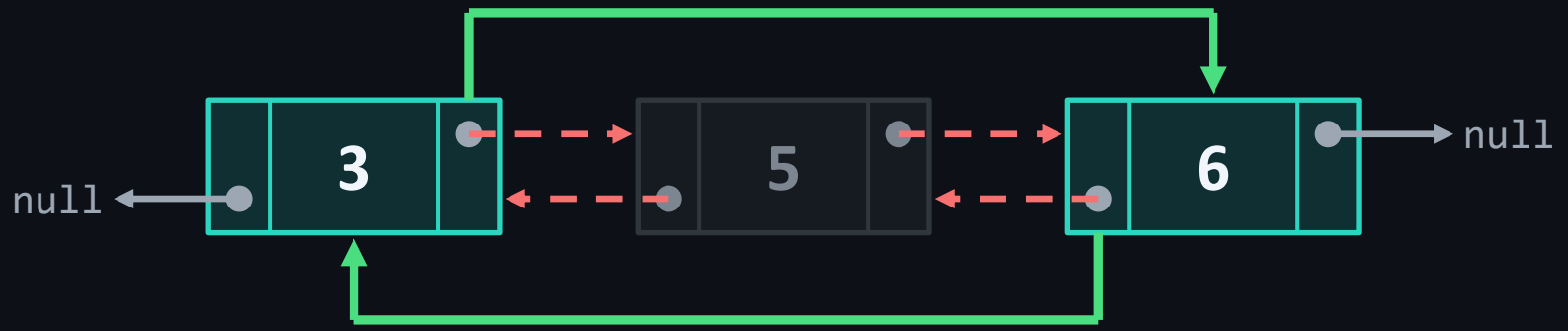
Lista doble: add y remove

add(9)



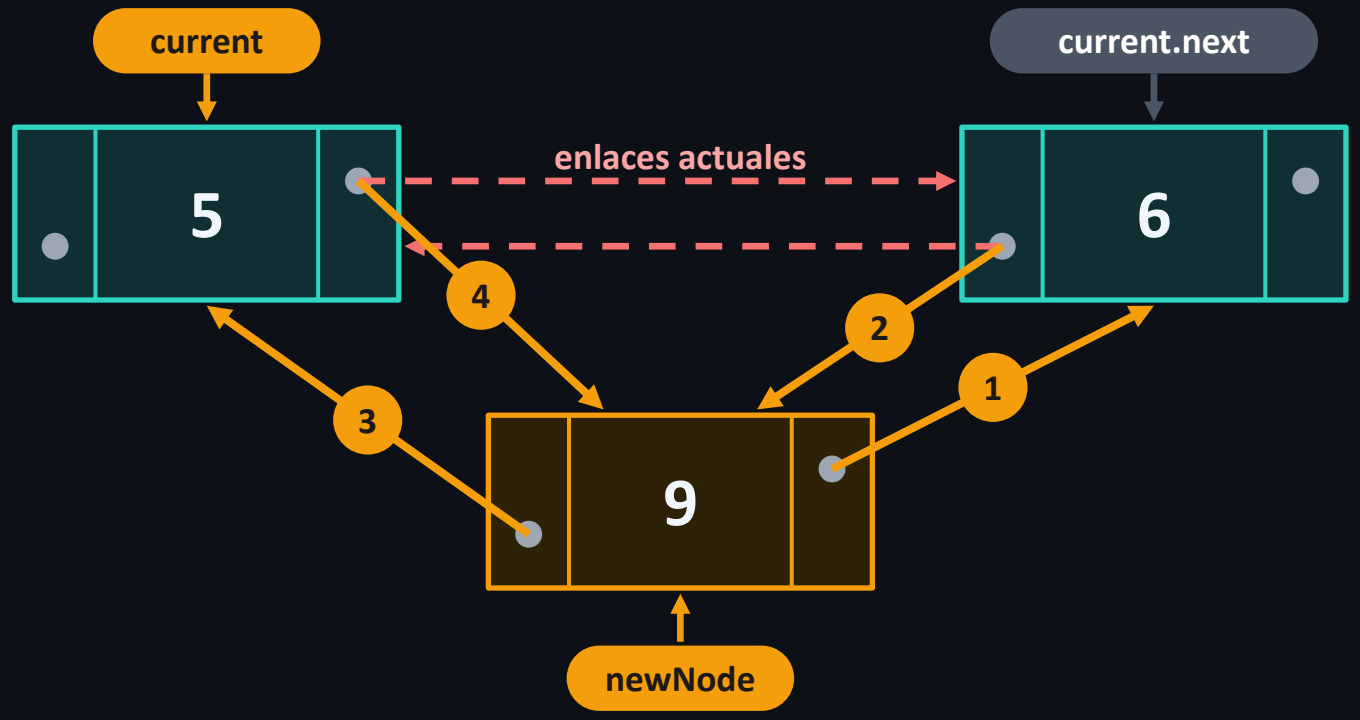
newNode.prev = current; current.next = newNode;

remove(5)



current.next = current.next.next; current.next.prev = current;

Lista doble: insertar entre dos nodos



Las cuatro asignaciones, en orden:

```
newNode.next = current.next;   // 1
current.next.prev = newNode;   // 2
newNode.prev = current;        // 3
current.next = newNode;        // 4
```

El orden importa: el 4 sobrescribe `current.next`, que el 1 y el 2 necesitan leer; el 3 puede ir en cualquier lugar.

En la lista simple bastan **2** enlaces; en la doble son **4**. **Dibuje** antes de escribir el código.

// lista doble

Lista doble: nodo, add y remove

Nodo con prev y next

```
class Node {
    int value;
    Node next;
    Node prev;

    Node(int value) {
        this.value = value;
    }
}
```

add: enlazar al final

```
current.next = newNode;
newNode.prev = current;
```

remove: nodo intermedio

```
current.next = current.next.next;
if (current.next != null) {
    current.next.prev = current;
}
size--;
```

> Al eliminar hay que actualizar **dos** referencias: `next` del anterior y `prev` del siguiente.

// lista circular

Lista enlazada circular



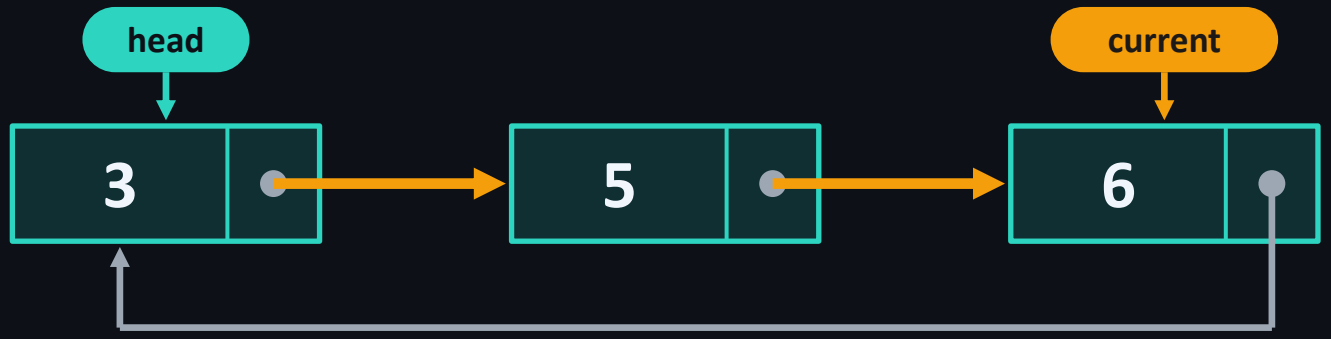
El último nodo apunta al **primero**: la lista no tiene final ni principio.

Aplicabilidad: lista de reproducción de música o lista de tareas pendientes.

// lista circular

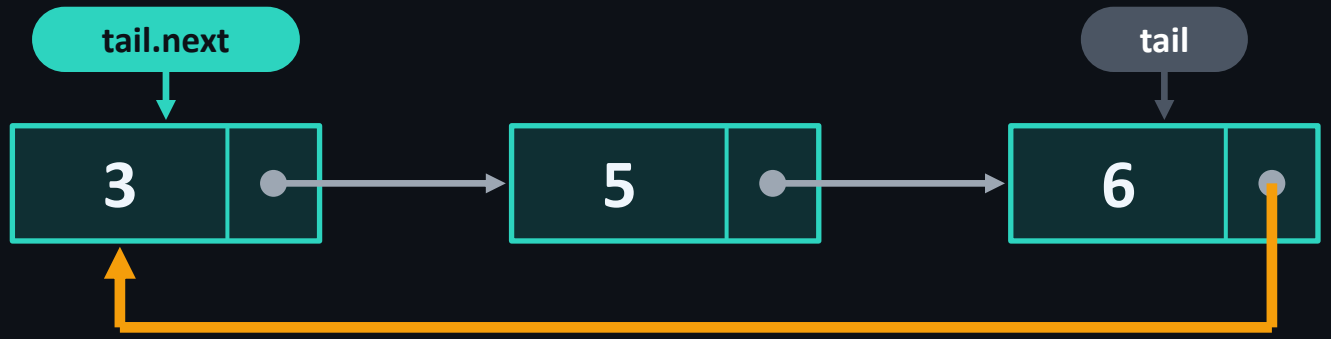
Lista circular: ¿guardar head o tail?

Solo head



Insertar al final o quitar el primero: **recorrer** hasta el último nodo

Solo tail

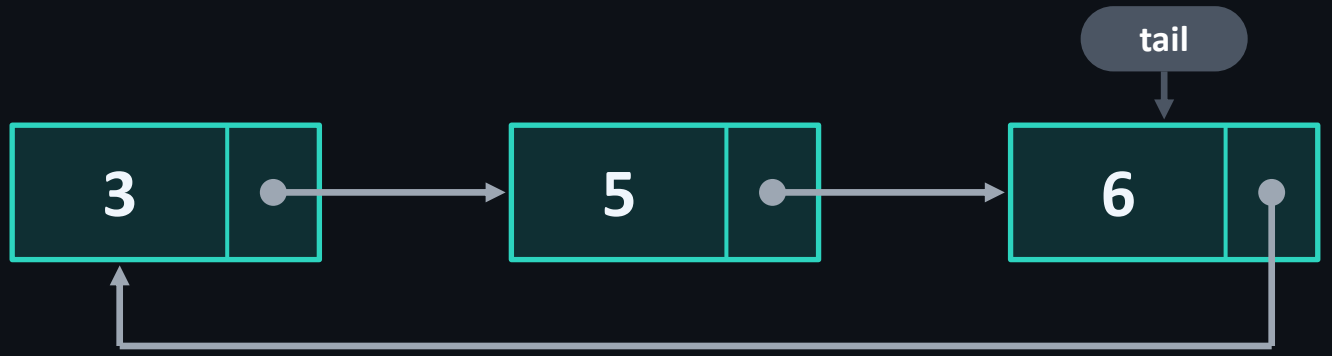


`tail` es el último y `tail.next` el primero: **sin recorrer**

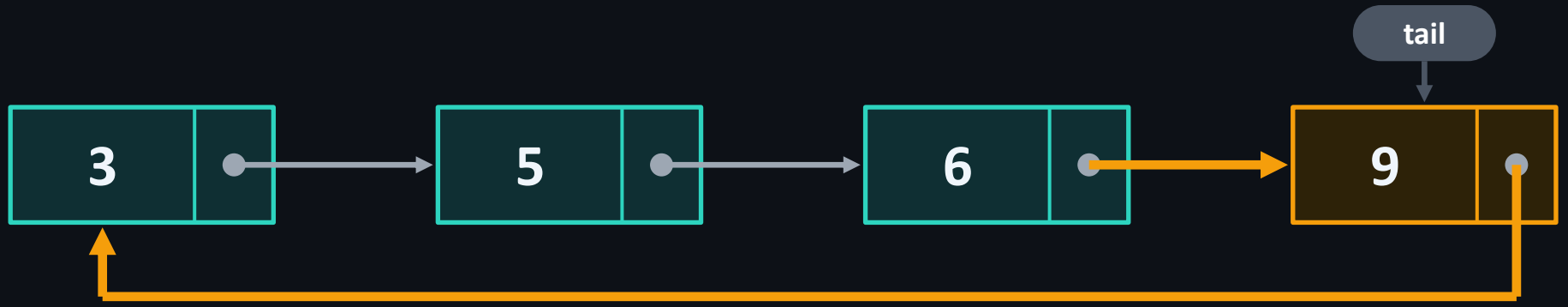
// lista circular

Circular con tail: agregar sin recorrer

Antes



add(9)



newNode.next = tail.next; tail.next = newNode; tail = newNode;

// lista circular

Lista circular: add con tail

add

```
public boolean add(int element) {
    Node newNode = new Node(element);
    if (tail == null) {
        newNode.next = newNode;
        tail = newNode;
    } else {
        newNode.next = tail.next;
        tail.next = newNode;
        tail = newNode;
    }
    size++;
    return true;
}
```

Lista vacía

El nodo se apunta a sí mismo y pasa a ser `tail`.

Lista con nodos

`newNode.next = tail.next` (el primero); luego `tail = newNode`.

`contains` y `remove` recorren desde `tail.next` hasta volver a `tail`.

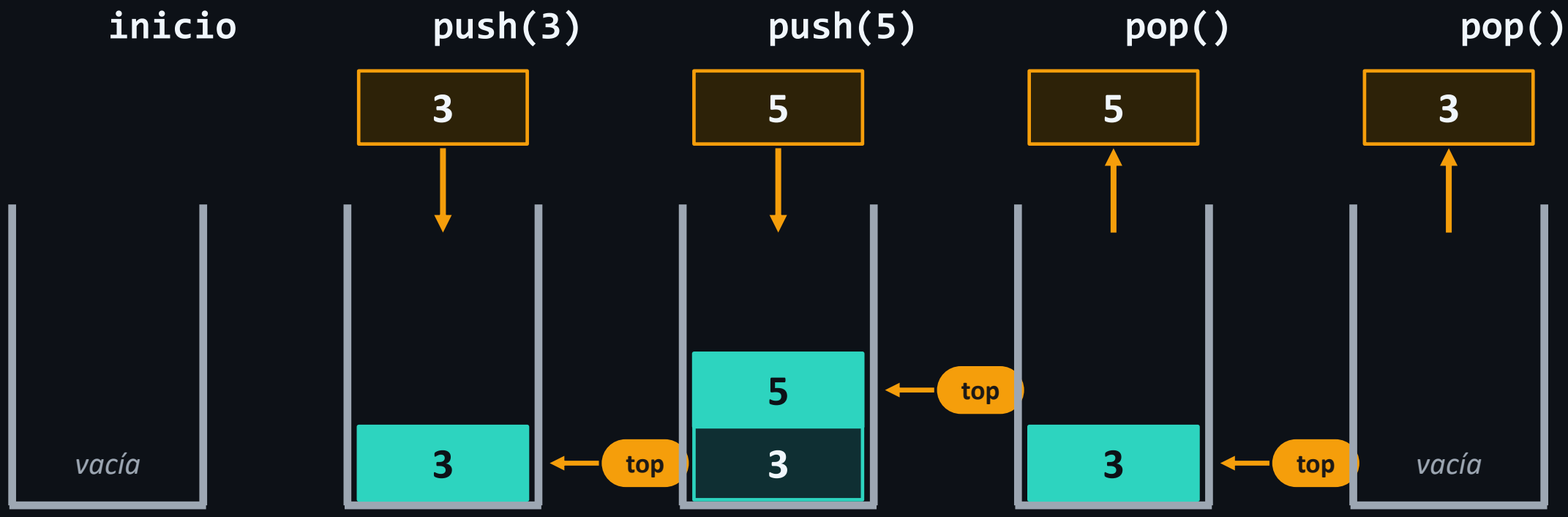
// lista circular

Comparación de implementaciones de lista

Implementación	Acceso	Insertar / eliminar	Uso de memoria
ArrayList	Rápido	Costoso	Siempre ocupa todo el espacio; sin overhead
Lista simple	Lento	Rápido	Solo lo que usa; overhead de una referencia
Lista doble	Lento	Rápido	Solo lo que usa; overhead de dos referencias
Lista circular	Lento	Rápido	Como la simple; sin final ni principio

Acceso por posición: **ArrayList**. Inserciones y eliminaciones frecuentes: **listas enlazadas**.

Pila: último en entrar, primero en salir (LIFO)



Solo se accede por el **tope**: el último elemento apilado es el primero en desapilarse.

Dónde se usa: procesadores, compiladores (interpretan expresiones) y muchos algoritmos.

TDA Pila: tres operaciones

1

Apilar (push)

Añade un elemento a la pila.

void push(int element)

2

Desapilar (pop)

Elimina el último elemento apilado.

int pop()

3

Tope (top)

Ve el elemento de la cima sin desapilarlo.

int top()

> Como las listas, una pila se implementa con **arreglos** o con **listas enlazadas**.

Pila con arreglo: el índice top



Pila con arreglo: push y pop

push

```
private int[] stack;
private int top;  // -1: pila vacía
private int size;

public void push(int element) {
    if (top == size - 1) {
        System.out.println("Stack Overflow");
    } else {
        stack[++top] = element;
    }
}
```

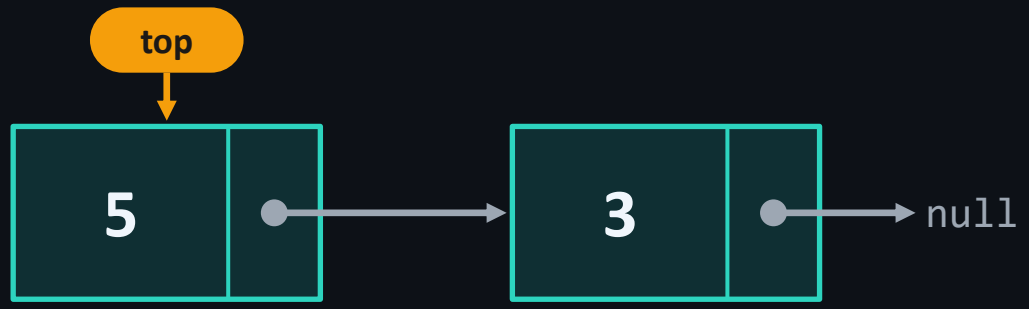
pop

```
public int pop() {
    if (top == -1) {
        System.out.println("Stack Underflow");
        return -1;
    } else {
        return stack[top--];
    }
}
```

> `++top` incrementa **antes** de escribir; `top--` decrementa **después** de leer.

Pila con lista enlazada: top es la cabeza

Antes



push(7)



push: `newNode.next = top; top = newNode;`

pop: `top = top.next;` y se devuelve el valor del nodo.

Pila con lista enlazada: push y pop

push y pop

```
public void push(int element) {
    Node newNode = new Node(element);
    newNode.next = top;
    top = newNode;
}

public int pop() {
    if (top == null) {
        System.out.println("Stack Underflow");
        return -1;
    }
    int element = top.data;
    top = top.next;
    return element;
}
```

Nodo como clase interna privada

```
private class Node {
    int data;
    Node next;

    public Node(int data) {
        this.data = data;
    }
}
```

Pregunta para la clase

¿Imprimir «Stack Underflow» y devolver -1, o lanzar una excepción?

Cola: primero en entrar, primero en salir (FIFO)



// colas

TDA Cola: tres operaciones

1

Encolar (enqueue)

Añade un elemento al final de la cola.

void enqueue(int element)

2

Desencolar (dequeue)

Quita el elemento del frente.

int dequeue()

3

Frente (front)

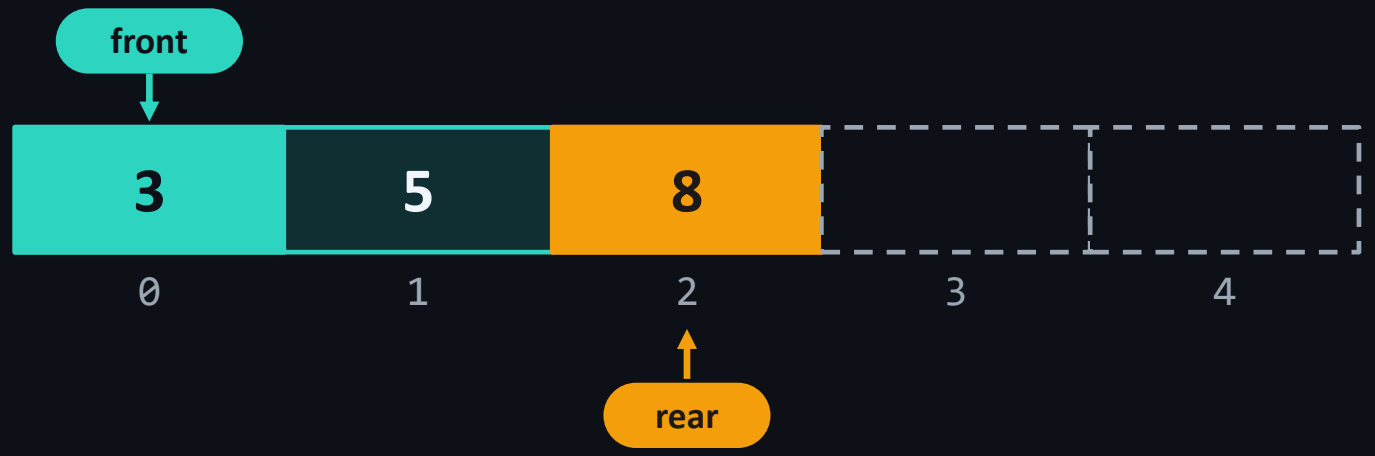
Ve el elemento del frente sin quitarlo.

int front()

Dónde se usa: colas de impresión y entrada del teclado del sistema operativo. Se implementa con **arreglos** o con **listas enlazadas**.

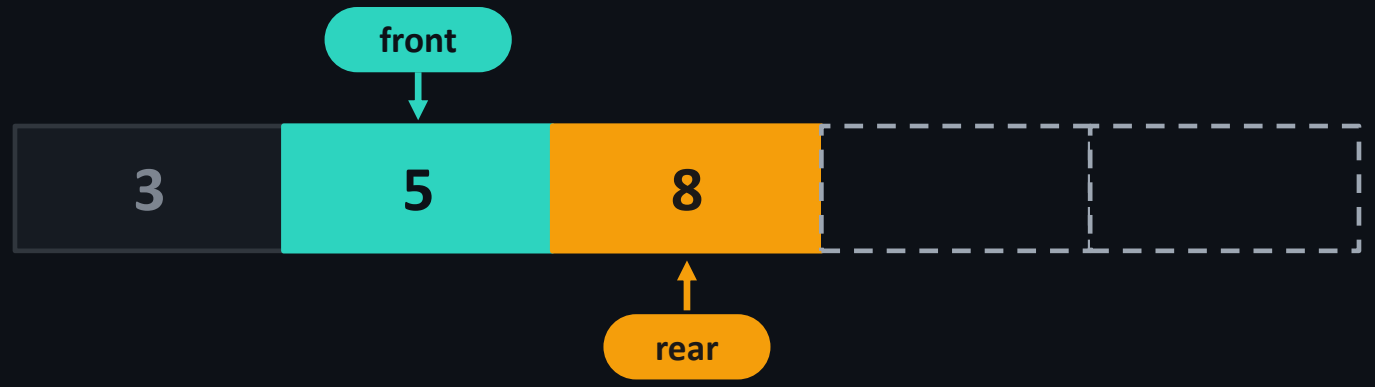
Cola con arreglo: front y rear

3 × enqueue




```
front = 0
rear = 2
```

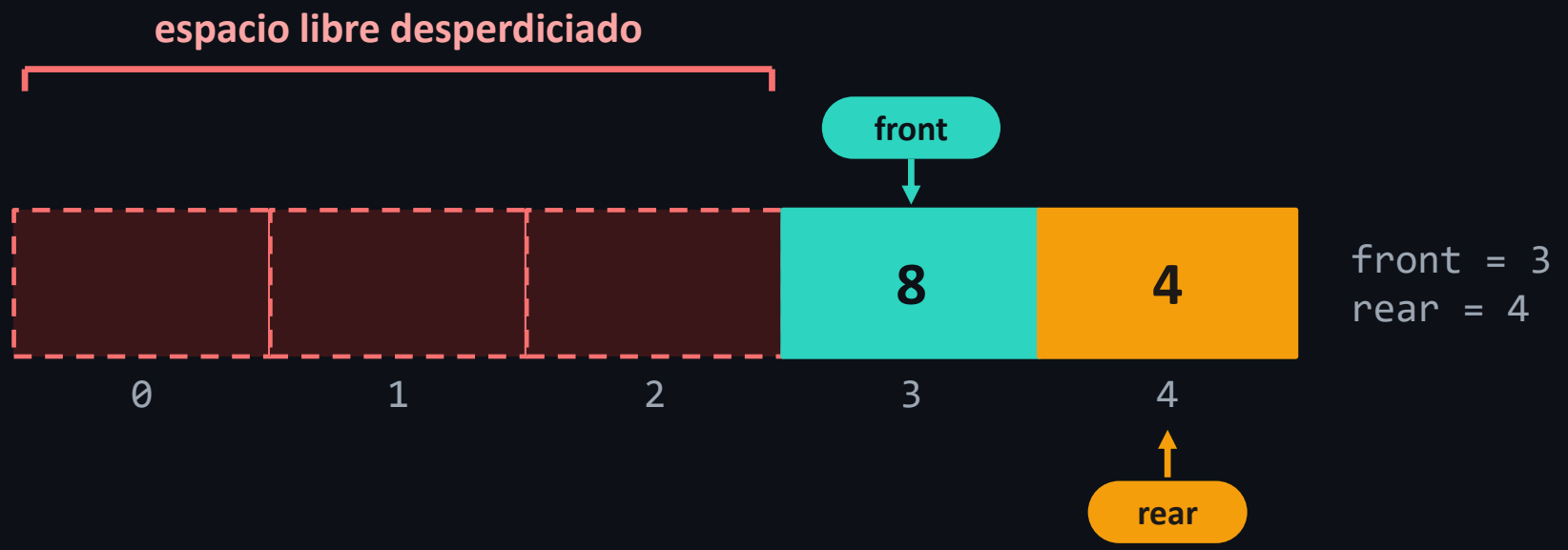
dequeue()




```
front = 1
rear = 2
```

El problema del «drifting»

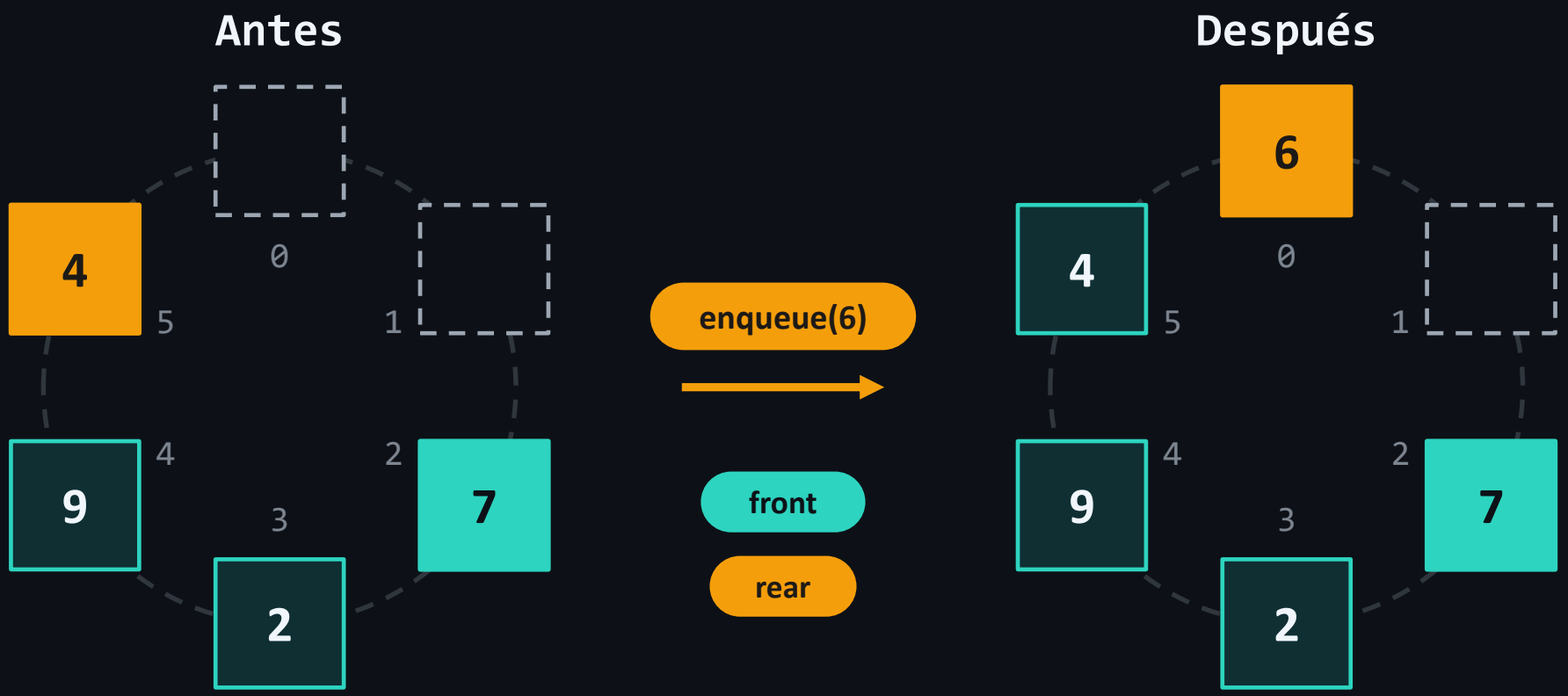
Tras muchas operaciones



`rear == size - 1`: «Queue Overflow» aunque haya 3 posiciones libres al inicio.

Soluciones: desplazar todo el arreglo en cada `dequeue` cuesta **O(n)**; mejor **envolver**: la cola circular.

Cola circular: el final da la vuelta



Cola circular: enqueue y dequeue

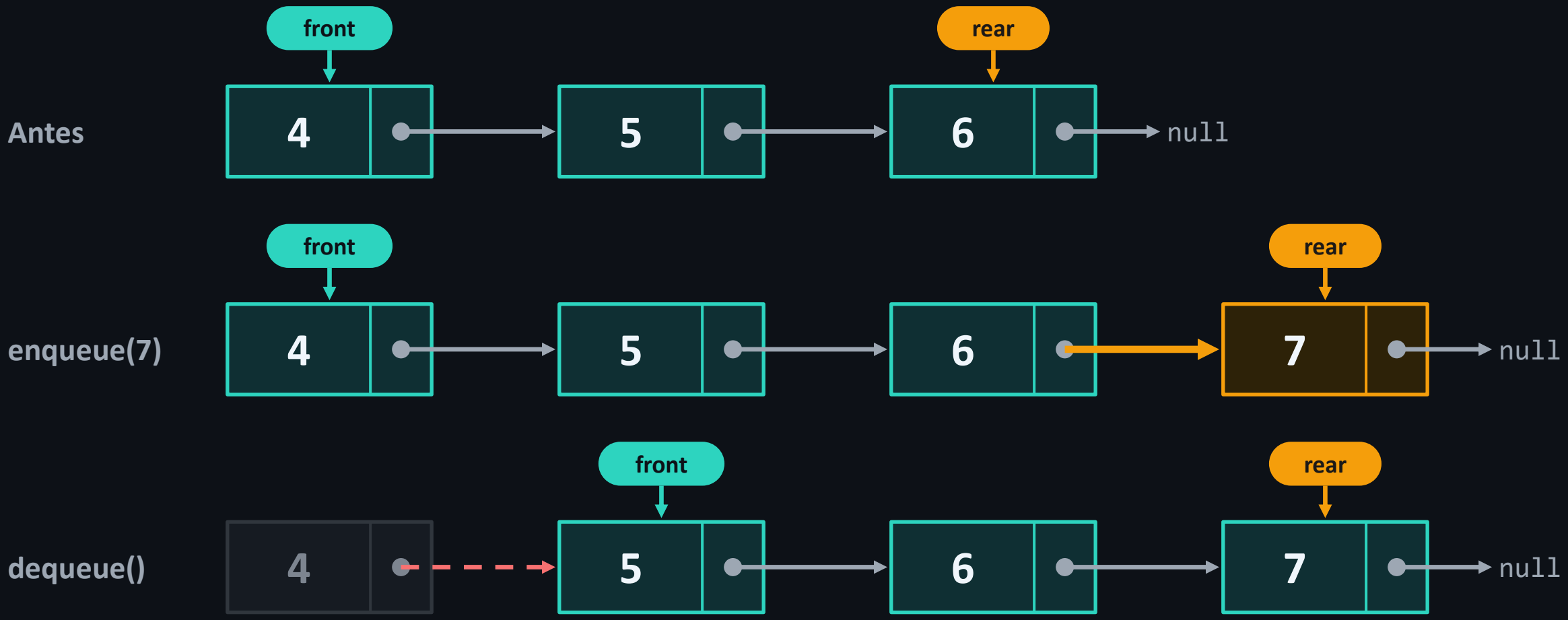
enqueue

```
public void enqueue(int element) {
    if ((rear + 1) % size == front) {
        System.out.println("Queue Overflow");
    } else {
        if (front == -1) {
            front = 0;
        }
        rear = (rear + 1) % size;
        queue[rear] = element;
    }
}
```

dequeue

```
public int dequeue() {
    if (front == -1) {
        System.out.println("Queue Underflow");
        return -1;
    }
    int element = queue[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % size;
    }
    return element;
}
```

Cola con lista enlazada: front y rear



Cola con lista enlazada: enqueue y dequeue

enqueue

```
public void enqueue(int element) {
    Node newNode = new Node(element);
    if (rear == null) {
        front = newNode;
        rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
}
```

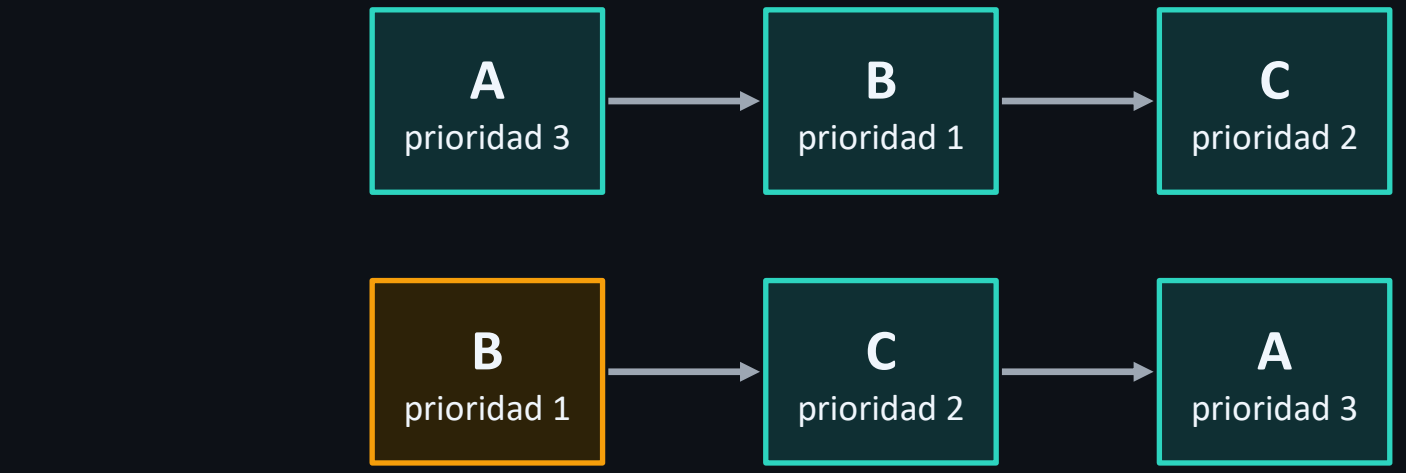
dequeue

```
public int dequeue() {
    if (front == null) {
        System.out.println("Queue Underflow");
        return -1;
    }
    int element = front.data;
    front = front.next;
    if (front == null) {
        rear = null;
    }
    return element;
}
```

`rear` evita recorrer la lista: `enqueue` toma tiempo constante.

Si la cola queda vacía, `rear` también se pone en `null`.

Cola de prioridad: sale primero el más prioritario



Ascendente: el menor valor numérico tiene mayor prioridad.

Descendente: el mayor valor numérico tiene mayor prioridad.

// colas de prioridad

TDA Cola de prioridad

Interfaz

```
public interface IPriorityQueue {
    void enqueue(int element, int priority);
    int dequeue();
    int front();
}
```

Implementaciones

Con arreglos, listas enlazadas o montículos (heaps). Aquí: lista enlazada ordenada; el heap se ve más adelante.

Encolar (enqueue)

Inserta según la prioridad, reorganizando si es necesario.

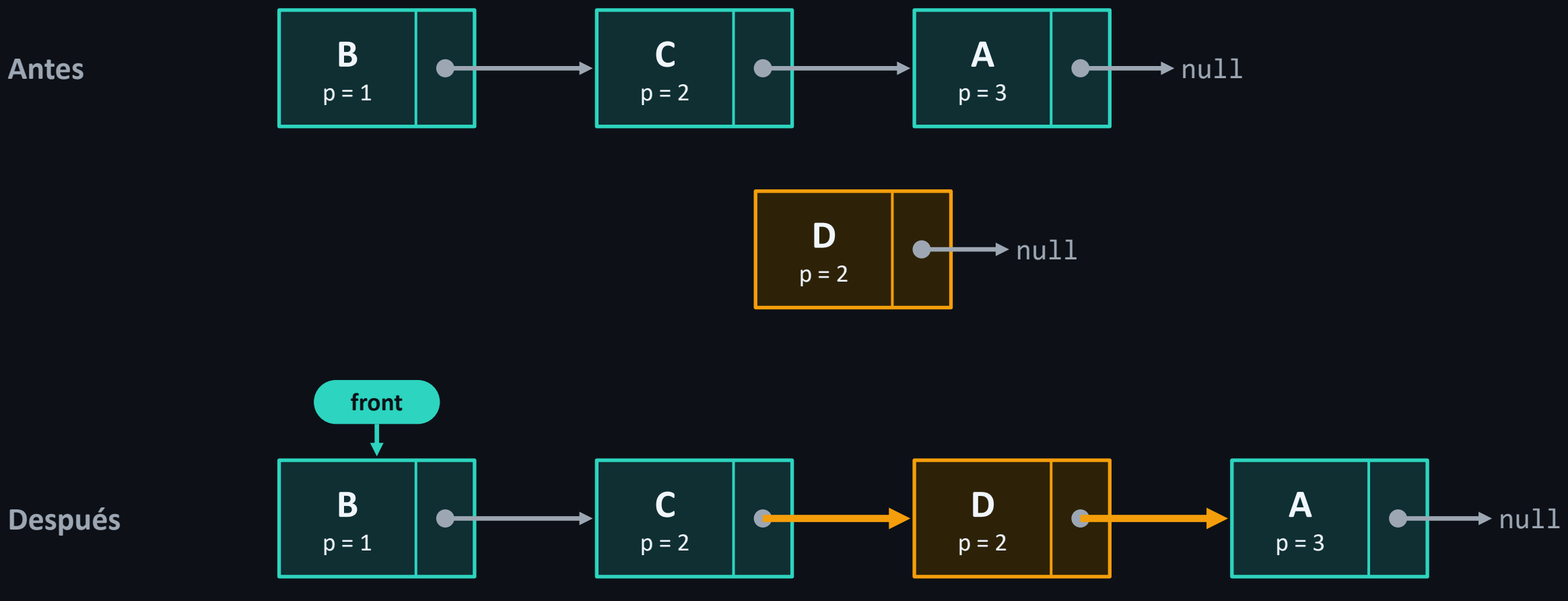
Desencolar (dequeue)

Elimina primero el de mayor prioridad.

Frente (front)

Ve el de mayor prioridad sin quitarlo.

Lista ordenada: enqueue(D, 2)



Cola de prioridad: enqueue ordenado

Cola vacía o prioridad mayor que front

```
Node newNode = new Node(element, priority);
if (front == null) {
    front = newNode;
    rear = newNode;
} else if (priority < front.priority) {
    newNode.next = front;
    front = newNode;
}
```

Caso general: buscar la posición

```
Node current = front;
while (current.next != null
       && current.next.priority <= priority) {
    current = current.next;
}
newNode.next = current.next;
current.next = newNode;
if (newNode.next == null) rear = newNode;
```

`<=`: un elemento nuevo va **después** de los que tienen su misma prioridad.

`dequeue` solo quita la cabeza: `front = front.next`.

// generics

¿Y si no es un entero?

Una clase por tipo

Una pila para `String`, otra para `double`…: no es eficiente ni escalable.

Usar `Object`

Se pierde el tipo específico: hay que hacer «casting» y pueden ocurrir errores en ejecución.

Generics

Clases, interfaces y métodos con un tipo que se especifica al crear la instancia.

Generics aplica a **cualquier** estructura de datos, no solo a las lineales.

// generics

Una clase genérica, muchos tipos

Lista genérica

```
public class LinkedList<T> {
    private Node<T> head;

    private static class Node<T> {
        private T data;
        private Node<T> next;

        public Node(T data) {
            this.data = data;
            this.next = null;
        }
    }
    // add(T data) y demás métodos
}
```

Instanciación con distintos tipos

```
LinkedList<String> list = new LinkedList<>();
list.add("Hello");
list.add("World");

LinkedList<Integer> numbers = new LinkedList<>();
numbers.add(1);

LinkedList<Double> decimals = new LinkedList<>();
decimals.add(3.14);
```

T es un tipo

Se reemplaza por `String`, `Integer`, `Double`… al crear la instancia.

Generics con Comparable

Elementos comparables entre sí

```
public class LinkedList<T extends Comparable<T>> {
    // ...

    private class Node<T extends Comparable<T>> {
        public int compareTo(T other) {
            return this.data.compareTo(other);
        }
    }
}
```

Comparable

Fuerza a que los elementos de tipo `T` sean comparables entre sí.

Tipos propios

`Persona`: comparar por cédula. `Estudiante`: por número de carné.

> Para un tipo **personalizado**, implemente `Comparable` con reglas de comparación que tengan sentido para la aplicación.

// cierre

Ahora usted

Resuelva en papel y compare con su compañero:

A · Rastree una pila

`push(4)`, `push(1)`, `pop()`, `push(7)`. ¿Qué contiene y cuál es el tope?

B · Rastree una cola

`enqueue(4)`, `enqueue(1)`, `dequeue()`, `enqueue(7)`. ¿Qué hay y quién está al frente?

C · Cola circular

`size = 5`, `front = 2`, `rear = 4`. Tras `enqueue(9)`, ¿cuánto vale `rear`?

D · ¿Qué estructura?

Deshacer cambios, fila de un banco e historial del navegador.

// cierre

Resumen

- **Arreglos y matrices:** posiciones contiguas, acceso en tiempo constante, tamaño fijo.
- **TDA y listas:** el TDA fija las operaciones; arreglo o nodos cambian el costo.
- **Pilas (LIFO):** `push`, `pop` y `top`, con arreglo o con lista enlazada.
- **Colas (FIFO):** `enqueue`, `dequeue` y `front`; la cola circular evita el «drifting».
- **Colas de prioridad:** sale primero el elemento de mayor prioridad.
- **Generics:** una sola implementación para cualquier tipo de dato.

Referencia: GeeksforGeeks, Introduction to Arrays. https://www.geeksforgeeks.org/introduction-to-arrays-data-structure-and-algorithm-tutorials/